

What Is a World Model Modeling? From Predicting the Future to Reusing Experience

Ying Wen

Shanghai Jiao Tong University
Shanghai Innovation Institute

ying.wen@sjtu.edu.cn

May 7, 2026

Abstract

World models span transition prediction, video generation, robotic action modeling, and language-agent planning, but predictive fidelity alone does not establish reusable competence. This paper introduces the Tri-World Model as a functional distinction among an Interactive Objective World, a Subjective World Model, and an External Knowledge World. The central hypothesis is that a bounded agent benefits from selecting future information according to controllability, failure recovery, and reuse under its current goal. A Capability Compounding Coefficient is proposed to evaluate whether verified experience reduces the cost of later tasks. The discussion connects this framework to scientific measurement, robotics, and digital agents, and states matched-budget tests under which an explicit subjective-model interface would be unnecessary. Existing minimal gridworld evidence does not establish general deployment efficacy.

Keywords: world-models, llm-agents, reinforcement-learning, embodied-ai

When people hear the term world model, the first intuition is often a model that can simulate the world. It might predict the next frame of a game, generate a short future video, imagine how an object will move before a robot acts, or predict what will happen after an action in reinforcement learning.

That intuition is not wrong. World models are indeed related to predicting the future. But if we only understand them as bigger simulators, we miss the more important problem: for an agent that must act, prediction is only the first step. The harder question is which future information the agent should keep, which information it should discard, and how one experience can become reusable capability later.

This essay first clarifies the term world model. Today, different people use the term for different implementations under a broad idea. State prediction in reinforcement learning, future video generation, robot state-action models, and long reasoning, planning, and memory in agent systems all fit into this broad frame. They all process future-related knowledge, but they differ in object, condition, and use. [Sutton [1]; Ha and Schmidhuber [2]; Schrittwieser et al. [3]; Hafner et al. [4]][5–7][8–10]

Another easily underestimated thread is physical signals and scientific world models. Many AI systems learn from worlds that humans have already written down, photographed, compressed, and labeled. But before the real world enters a model, it has often passed through sensors, sampling, calibration, display, compression, and human naming. Spectra, vibration, current, temperature, remote sensing, experimental instrument readings, continuous dynamical systems, and scientific simulations remind us that world models do not only predict futures in text and RGB images. They may also learn regularities from measurement streams with noise, instrument bias, and time scale. [phy [11]][12–18]

On that basis, we introduce the Tri-World Model. It separates the problem into three roles: how the environment gives feedback, how the agent forms its current judgment, and how experience becomes external knowledge. To keep the discussion readable, I will name them up front: the Interactive Objective World (W1), the Subjective World Model (W2), and the External Knowledge World (W3).

The narrower claim is this: what next-generation agents may really lack is not a more complete copy of the world, but an internal world model that selects controllable futures. That is the Subjective World Model (W2). Later we will also use an evaluation concept called the Capability Compounding Coefficient (CCC). CCC asks not only whether a system solved this task, but whether one verified experience reduces the cost of future tasks. The second half of this essay makes the implications more concrete for training methods, trajectory logs, data collection, and system design across domains. In embodied AI and robotics in particular, it leads to a training target that is different from current vision-language-action models and world action models: the system should not only become better at outputting actions or generating futures, but should explicitly learn object state, task stage, affordance, failure recovery, and experience reuse.

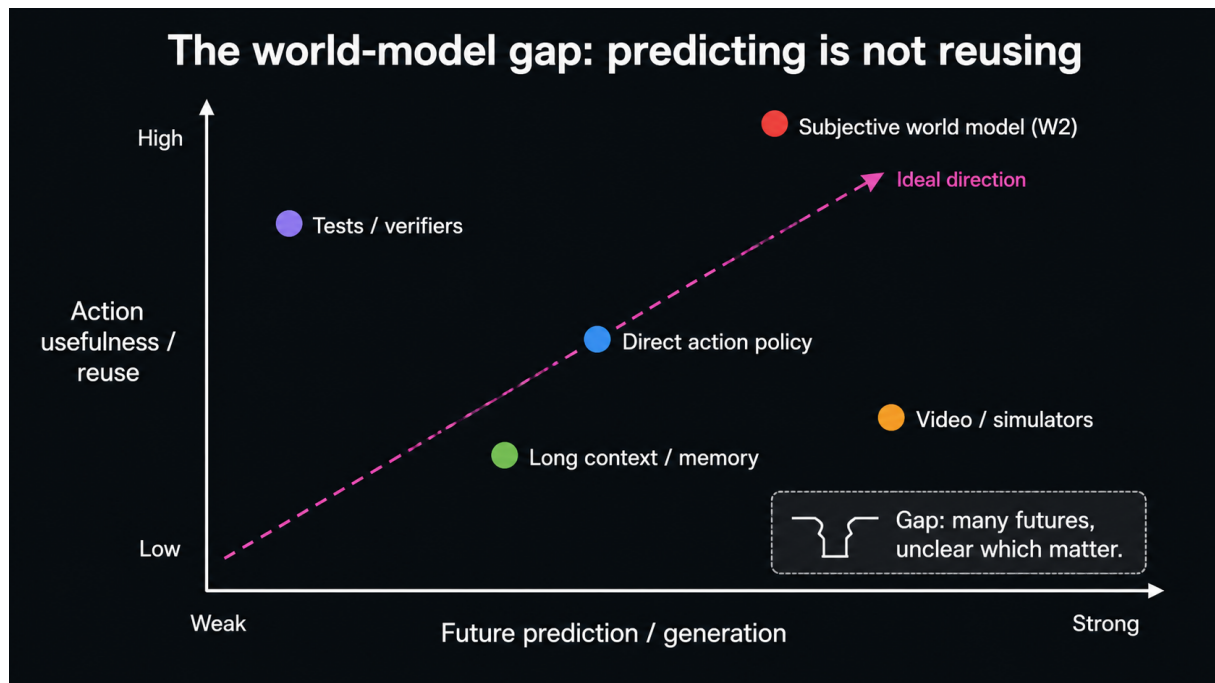


Figure 1: The world-model gap: predicting is not reusing.

1 Start With Familiar World Models

In today’s technical context, world models usually mean one of four things.

The first comes from reinforcement learning. An agent acts in an environment, and the model predicts the next state, reward, or value. It answers the question: if I take this action now, what will the environment do next? This kind of model matters for planning because the agent can simulate internally before acting in the real environment. Dyna, World Models, MuZero, and Dreamer all belong to this lineage. [1–4]

The second comes from video and spatial generation. A model generates future frames from text, images, or video. It answers the question: what might this scene look like next? This makes the model feel like a world simulator because the future is visually intuitive. Work such as V-

JEPA and Sora makes the relationship between world models, visual futures, spatial consistency, and physical or digital simulation more explicit. [5–7]

The third comes from robotics and embodied intelligence. One nearby route is the World Action Model (WAM), which tries to put actions and future images or future states into the same model. It answers the question: under this task, how should I move, and what might the world look like after I move? This is important because it pushes world modeling toward bodies, objects, action consequences, and failure recovery. It also makes one distinction clearer: training action outputs or future images is not the same as training an internal judgment layer that keeps object state, task stage, body constraints, and recovery cues. [19–23]

The fourth comes from large language model agents. Here the world is not necessarily physical space. It may be a browser, codebase, terminal, document, or tool system. The model stores plans, tool calls, execution traces, test results, and repair experience. It answers the question: how can prior execution experience and external knowledge help with future tasks? ReAct, Reflexion, Voyager, and related agent work show how language models can use tools, feedback, reflection, and skill libraries to turn one execution into cues for later action. [8–10]

All four can be called world models, but they are not the same thing.

Context	What world model usually means	Future information it predicts or keeps	What it does not automatically solve
Reinforcement learning	State, reward, or value after action	How the environment changes	Open tasks, long-term experience, external knowledge reuse
Video generation	Future frames or spatial structure	What the scene may look like next	Which variables are controllable and which details matter for action
Embodied AI / robotics	Vision/body state to action or action effect	How to move next and how objects change	Failure cause, recovery path, task stage
LLM agents	Memory, plans, tool traces, tests, skills	Which past signals help future tasks	Whether old memory is stale or current state still matches

The point of this table is not taxonomy. It is to remove a misunderstanding. Most uses of world model today are different implementations under a broad idea. They share an abstract problem: under some condition, predict or retain information useful for the future. But the implementations can be very different.

So when we ask what kind of world model next-generation agents need, we should not only ask whether a model can predict the future. We should also ask whether the predicted future is actionable, goal-relevant, useful for recovery, and capable of becoming reusable experience.

Takeaway: Do not turn successful examples into input-output pairs only. For action systems, it is more useful to put the goal, action, predicted effect, actual feedback, failure repair, and later reuse into the same trajectory.

2 Prediction Is Not Enough

There is an easy distinction to miss: being able to predict the future is not the same as knowing which future information to keep.

A video model may generate a realistic future kitchen scene, but a robot does not need every reflection and texture in that image. It needs to know where the cup is, whether it can be grasped, whether the arm will hit an obstacle, and how to recover after failure. A browser

agent may save a complete operation log, but after the page refreshes, an old button position may no longer be valid. A code agent may save an entire repair trace, but the valuable parts are which test verified which hypothesis, which patch route failed, and which modules have hidden coupling.

These examples are directly related to familiar world models. They do not say prediction is useless. They say there is too much future information. For an action system, the key is not to predict more and more of the future, but to select the future information that matters most under limited capacity.

That is the motivation of this essay. The world model we care about is not only a future predictor. It is a mechanism for selecting future knowledge.

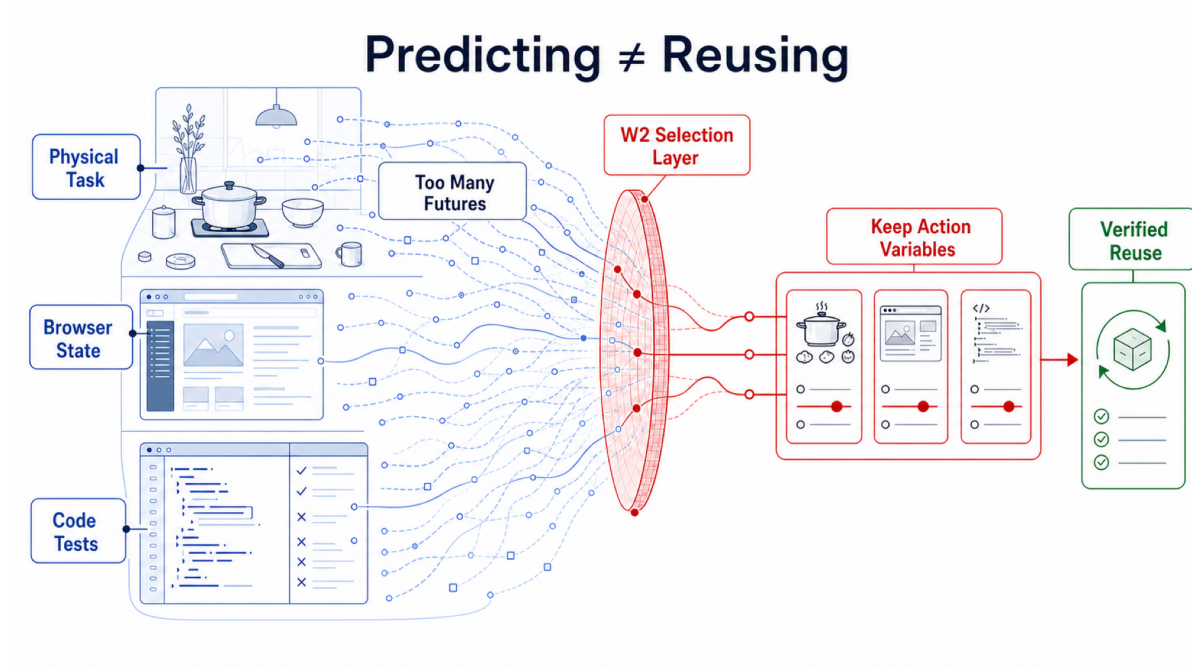


Figure 2: There are too many possible futures; W2 selects action variables, failure risk, effects, and reusable experience under limited capacity.

3 Our Definition of World Model

We can define it plainly:

A world model is an agent's prediction or compression of useful future information under its current situation, current goal, possible actions, and uncertainty.

The current situation includes what the agent has observed, what it has already done, what tools and permissions it has, and what state the page or physical environment is in. More strictly, the agent usually does not observe the world itself, but some measurement of it: images, logs, API returns, sensor readings, experimental records, or descriptions written by humans. Every measurement has a sampling method, noise, latency, calibration, and bias. The current goal determines what matters. Picking up a cup and cleaning a table require different variables. Fixing a bug and refactoring code require different clues. Possible actions determine what the agent can change. It can click a button, run a test, move a robot arm, or ask a user; it may also be unable to change some background factors. Uncertainty reminds us that many key states are not directly visible and must be inferred through action and feedback.

The useful future information does not have to be the next frame. It can be next state, reward, value, action effect, failure risk, tool state, test result, reusable rule, skill, repair pattern, or even the fact that an old memory should no longer be trusted.

So the inputs are current situation, current goal, possible actions, and uncertainty. The output is useful future information. This definition is broad enough to compare reinforcement learning models, video models, robot models, and LLM-agent memory on the same map. But it naturally leads to a stricter question: in a big world, an agent cannot keep everything. What should it keep?

4 If You Cannot Keep Everything, What Should You Keep?

The world is large, and the agent has limited capacity. This sounds abstract, but it is concrete in real systems.

A codebase can contain thousands of files. A browser page contains text, buttons, pop-ups, state, and background requests. A robot camera sees countless pixels. A conversation history contains a lot of material that may or may not matter. Even a long context window cannot treat everything as equally important.

Ordinary compression may keep what best reduces prediction error. Ordinary memory may retrieve what looks similar to the current problem. Ordinary vision models may optimize visual coherence: what makes the future video more realistic.

For an agent that must act, those criteria are insufficient. It should prioritize information that is important for the current goal, that the agent can influence, that helps with recovery after failure, and that can later be verified and reused.

Call this actionable compression.

A city map gives a simple example. A complete map can show all roads, shops, and terrain. But if you are trying to catch a train, what you really need is your current location, a route to the station, which roads are closed, and which mode of transport can still get you there on time. The complete map is useful, but the current task needs a map folded around goal and action.

Agents are the same. They do not need an internal copy of the whole world. They need a map that knows what to look at.

Ordinary world models may care about	Acting agents need to care about
What may happen in the future	Which futures matter under the current goal
What the world will look like	Which states I can change
How to reconstruct more detail	Which details can be safely discarded
Whether the current policy succeeds	What failure looks like and how to recover
Whether this task is solved	Whether this experience reduces future cost

This selection problem is the core problem for the Tri-World Model.

5 Popper's Three Worlds as a Starting Point

The name Tri-World is not arbitrary. One inspiration is Karl Popper's theory of three worlds. Popper distinguished the world of physical objects and processes, the world of subjective experience and mental states, and the world of objective knowledge content. The third includes theories, problems, arguments, books, mathematical objects, and cultural artifacts. Once written down, they are no longer just a momentary thought inside one person's head; they can be inspected, criticized, inherited, and transformed by others. [Popper [24]][25]

This distinction is useful for AI. Today's agents also face three kinds of things: they must act in an environment changed by their actions; they must form a current internal judgment; and they must rely on external documents, code, tests, tools, skills, and memory. Many system failures happen because these three are mixed together. For example, an agent may treat old memory as current state, treat a document as verified action knowledge, or treat a model that generates future video as if it already knows how to act.

But this essay does not simply import Popper. Popper's Three Worlds is primarily a philosophical and epistemological framework. The Tri-World Model here is an operational framework for agent engineering. The Interactive Objective World (W1) includes not only the physical world, but also browsers, codebases, terminals, and social feedback. The Subjective World Model (W2) is not human consciousness; it is the agent's goal-conditioned, capacity-limited internal compression and judgment. The External Knowledge World (W3) is not only abstract theory; it includes knowledge artifacts that systems can read, write, test, execute, and reuse.

6 The Tri-World Model: Separate Environment, Internal Judgment, and External Knowledge

The Tri-World Model is not meant to create new terminology for its own sake. It separates three things that are often mixed together: an interactive environment, the agent's internal judgment, and reusable external knowledge.

The first layer is the Interactive Objective World (W1). If the agent's action can change it, it belongs to W1. A robot kitchen, browser page, codebase, command line, multi-agent collaboration environment, and interactive simulator can all be part of this layer. Its role is to provide feedback: whether an action took effect, whether the state changed, where the failure happened, and which constraints remain. For physical and scientific tasks, W1 also includes the measurement interface itself: sensors, instruments, sampling frequency, calibration state, unit system, and observation delay. Otherwise the model may mistake instrument bias for a law of the world.

The second layer is the Subjective World Model (W2). Subjective does not mean self-awareness. It means task-relevant, agent-perspective, capacity-limited compression. The same page, room, or codebase should be compressed into different states under different goals. W2 asks: what do I currently believe about the world? Which variables matter for the goal? What can I change? Where is the failure risk? What may happen after the next action?

The third layer is the External Knowledge World (W3). It includes language, code, tests, documents, rules, tools, skills, logs, source notes, and verifier outputs. It also includes equations, units, conserved quantities, scaling laws, causal graphs, experimental protocols, error models, and instrument calibration notes in scientific tasks. W3 is persistent, shareable, and composable. Once an experience is verified and written as a test, rule, skill, or scientific artifact, it becomes a knowledge artifact rather than a temporary trace.

The easiest confusion is between W2 and W3. Long context, retrieval, memory stores, skill files, and test cases are mostly W3 or W3 infrastructure. They are external knowledge, not current judgment. An old test may still be useful, or it may be stale because the code structure changed. A browser automation script may work for an old page but fail on a new page. W2 is responsible for deciding whether a W3 artifact still applies to the current W1 state.

In other words, the Tri-World Model does not say AI systems must be mechanically split into three modules. It says any agent that acts and learns over time must handle three roles: interact with the world, form a current judgment, and preserve reusable knowledge.

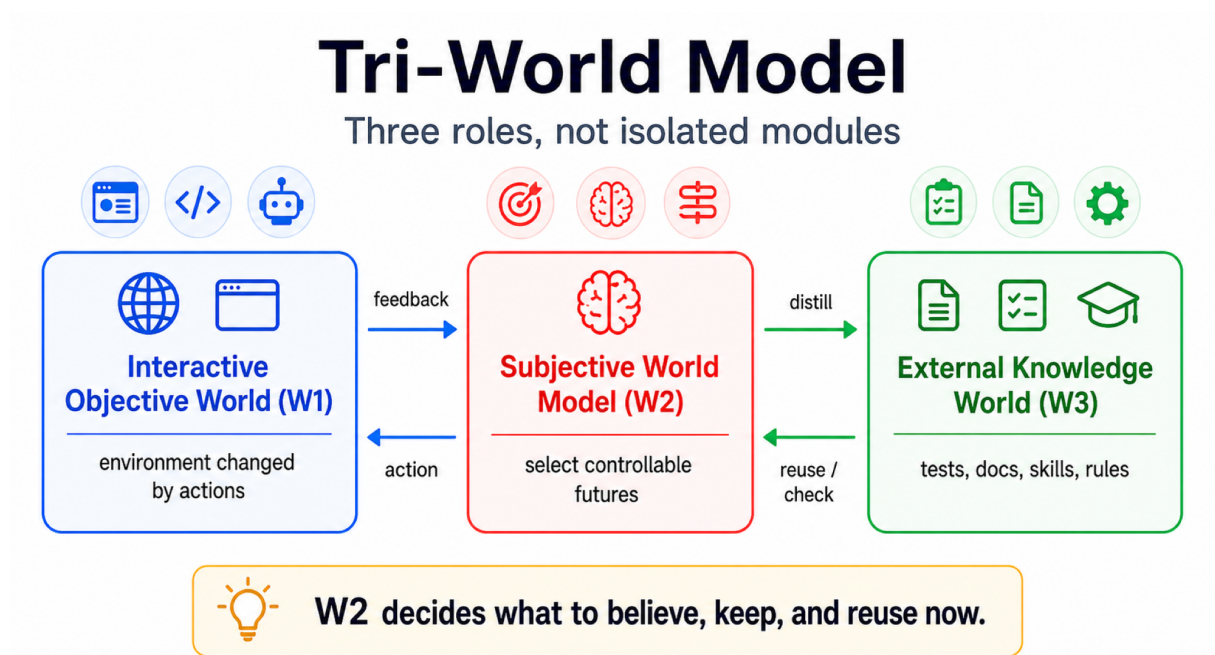


Figure 3: The three roles of the Tri-World Model: Interactive Objective World, Subjective World Model, and External Knowledge World.

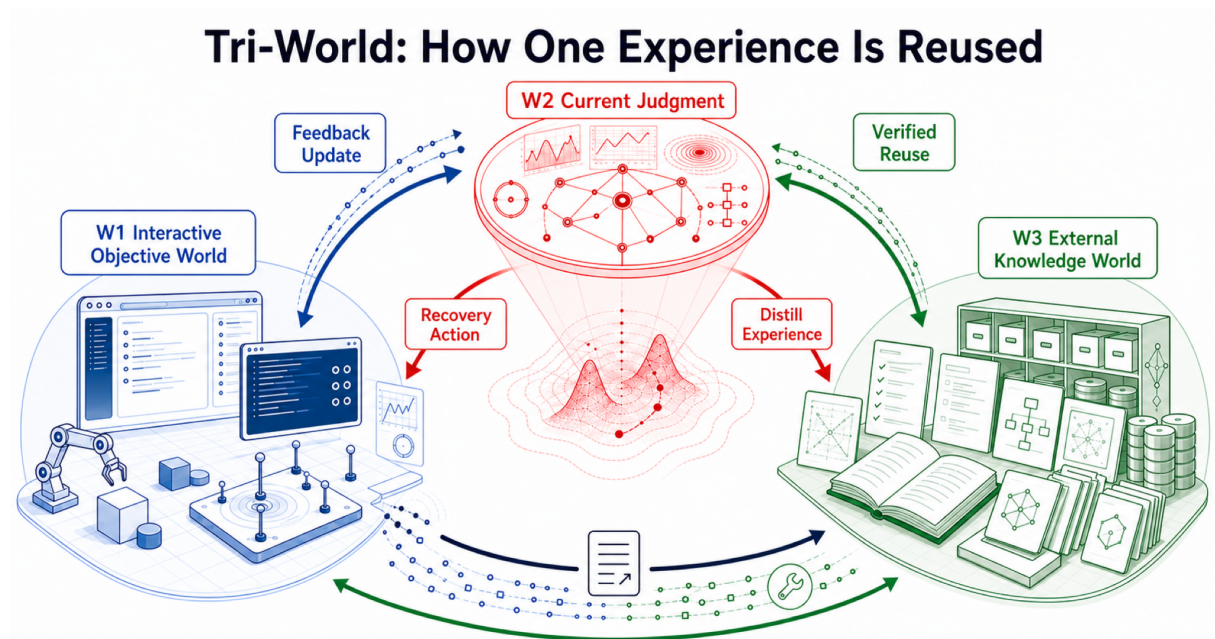


Figure 4: Tri-World is not a static taxonomy; it is an experience-reuse loop across W1 feedback, W2 judgment, and W3 distillation and retrieval.

7 Why the Subjective World Model Is the Key Missing Layer

Many current systems are already strong, but they are strong in different places.

Video and interactive generation strengthen W1 because they can generate or simulate future worlds. Agent runtimes, memory, tool protocols, and code tests strengthen W3 because they allow systems to store and call external knowledge. Robot policies and tool workflows strengthen direct execution because they can turn language, vision, or documents into actions.

All of this progress matters. But it can bypass the Subjective World Model.

The first bypass is a stronger simulator. If we can generate more realistic videos, larger interactive environments, and richer synthetic data, do we already have a world model? Partly. But the agent still has to know which simulation results to trust, which variables are relevant to the current goal, and which interventions can actually change outcomes. Simulating a world is not the same as knowing what to keep.

The second bypass is longer memory. If we save complete execution logs, tool calls, historical experience, and skill libraries, do we have intelligence? Again, only partly. Memory can be stale, polluted, or detached from current state. Without W2's scope and evidence checks, the External Knowledge World can become noise.

The third bypass is a direct action policy. If a model can output actions directly from language or vision, do we still need an internal world model? For many tasks, direct action may be enough. But once it fails, the system needs to know why it failed, which state changed, how to recover, and which parts of the experience remain useful later. Direct action can succeed without producing interpretable recovery or reusable experience.

Embodied robotics makes this issue larger. The strength of the VLA route is that it connects web-scale vision-language knowledge and robot demonstrations, allowing a model to output actions from images, instructions, and body state. WAM or WorldVLA-style routes go further by jointly modeling actions with future images or future states. These systems should be treated seriously as substrates and strong baselines, not dismissed.

Using today's large-model training as an analogy, VLMs, video models, and physical world models are closer to large-scale pretraining: they learn appearance, semantics, space, and dynamic priors. VLA, WAM, and demonstration trajectories are closer to action-oriented instruction tuning: they learn to produce reasonable actions or action-conditioned futures given instructions and observations. The Subjective World Model adds something closer to reinforcement-learning-style post-training: through real or high-quality simulated interaction, verifiers, failure feedback, and cross-task reuse, it teaches the system which variables are controllable, when to trust a model, how to recover after failure, and which experiences deserve to be distilled.

But there is a fundamental difference: the publicly visible training pressure in current VLA/WAM routes often focuses on whether actions resemble demonstrations or whether futures resemble real trajectories. The Subjective World Model additionally optimizes for which states are controllable, how to recover after failure, and whether experience can be reused later. If a robot only learns a plausible action for the current image but does not know whether the cup has been grasped, whether the drawer is truly open, which task stage it is in, or whether a gripper offset should trigger retry or strategy change, it may look good in familiar distributions but lack inspectable judgment under occlusion, layout shift, long-horizon tasks, and recovery.

Takeaway: VLA and WAM can be action substrates or training signals, but the Subjective World Model must also learn object state, task stage, affordance, body constraints, failure type, recovery action, and simulation trust.

Route	What it provides	Why it is not automatically W2
Stronger simulator	Richer future imagination	It may not know which futures matter for current goals and actions
Longer memory	More history and experience	It may not know whether memory still applies to current state
Vision-language-action model	Faster path from image/language to action	It may not expose object state, task stage, affordance, or recovery
World action model	Connects action with future images or states	If it only generates futures or actions, it may still lack reusable internal judgment
Higher benchmark score	Better final result	It may not reveal mechanism, transfer, or experience compounding

This is why W2 may be necessary. It is the middle layer that turns feedback from W1 and knowledge from W3 into actionable judgment.

The principle is simple:

A finite agent should spend representational capacity on futures that matter for the current goal and that the agent can influence.

That is controllability-aware compression. It does not compress everything and does not predict everything. It spends limited attention on actionable futures. More concretely, W2 repeatedly asks four questions: Is this relevant to the current goal? Can I influence it? Can it help me recover after failure? Can it become a rule, test, or skill later?

A browser agent should not remember all page text equally. It should remember controls related to the task, login state, error messages, whether the page refreshed, and whether the last click actually took effect. A robot should not only remember salient pixels or imitate the next action chunk. It should keep the target object, graspability, occlusion, arm state, task stage, and recovery action. A code agent should not save only a complete execution log. It should turn tests, failed patch routes, and module coupling into reusable knowledge artifacts. A scientific agent should not only save paper abstracts or experimental screenshots either. It should retain measurement conditions, variable definitions, error sources, intervenable parameters, counterfactual questions, and later-testable model hypotheses.

Once seen this way, W2 is not decorative. It is the selection mechanism required by a large world.

8 How to Tell Whether a System Really Has a Subjective World Model

We cannot judge by names. A system may not contain a module called W2 but may functionally satisfy the role. Conversely, a system may draw a W2 box but fail to improve recovery, transfer, and experience reuse.

We need behavioral evidence.

First, can it retain key state under limited capacity? The question is not whether it can remember everything, but whether it keeps the variables that actually affect the goal under task switching, distractors, and partial observation.

Second, can it predict action effects? After clicking a button, editing code, or moving an object, can it anticipate how the world should change and update when observation disagrees?

Third, can it recover from failure? Strong systems are not systems that never fail. They are systems that do not repeatedly hit the same failure. They should identify failure type, shorten recovery paths, and distill failure experience.

Fourth, can it turn experience into reusable external knowledge? A one-off success that does not reduce future task cost is closer to luck than knowledge accumulation. A verified artifact with scope and future cost reduction is a real knowledge asset.

Fifth, do these gains survive hidden tasks? Otherwise we may only be measuring benchmark memory, prompt tricks, or data leakage. Benchmarks such as ARC-AGI-3, OSWorld, and SWE-bench do not prove W2 by themselves, but they expose hidden rules, real software state, test verifiers, and future-task transfer in ways that make experience reuse measurable. [26–28]

Takeaway: Do not measure only one-task success rate. Measure actions on hidden tasks, repeated failure rate, recovery steps, human takeover, verification cost, and later reuse gain.

9 Capability Compounding Coefficient (CCC): Does Experience Compound Capability?

To measure whether experience is truly useful, we propose the Capability Compounding Coefficient (CCC).

The name sounds abstract, but the question is simple:

How much future cost does one verified experience save?

Ordinary benchmark scores ask whether the current task was solved. CCC asks whether the experience made future tasks cheaper. If a code agent writes a test that prevents ten useless future attempts on a similar bug, that is a gain. If a browser agent learns a workflow but page changes make that workflow misleading, the maintenance and error cost may exceed the gain.

CCC benefit	Plain meaning
Fewer actions	Fewer steps, retries, or dead ends next time
Fewer failures	Fewer repeated errors
Lower verification cost	Fewer useless tests, or faster discovery of the key verifier
Less human takeover	Fewer human corrections
Better transfer	Still useful after task, layout, or environment changes
Higher success	Not only faster, but more reliable

CCC cost	Plain meaning
Creation cost	How much interaction was needed to get this experience
Verification cost	How many tests, reviews, or verifiers were needed
Retrieval cost	How much effort future systems spend finding and using it
Maintenance cost	How stale, conflicting, or polluted knowledge is handled

CCC cost	Plain meaning
Misuse cost	How much damage occurs when the experience is used in the wrong place

CCC does not say storing more experience is always good. It requires that experience be verified, scoped, and shown to reduce future cost on hidden tasks. A system that simply stuffs all history into context does not necessarily have positive CCC. It may be bringing the trash back too.

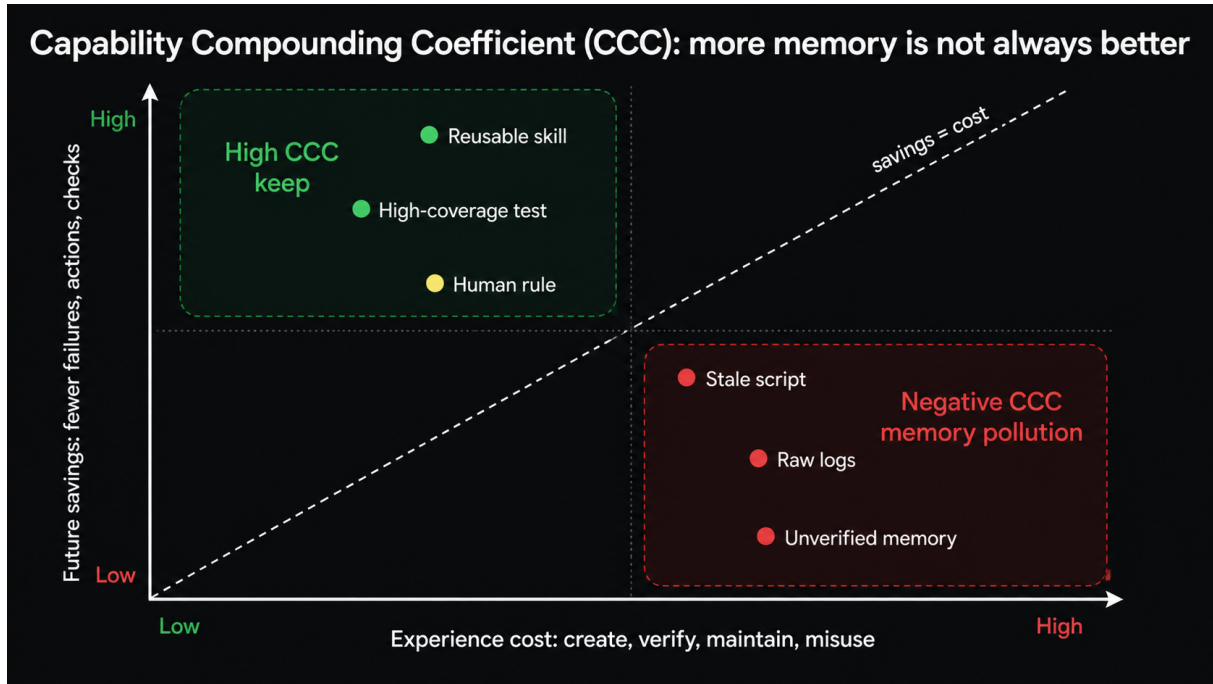


Figure 5: Capability Compounding Coefficient (CCC): high-benefit, low-cost experience should be distilled; high-cost, low-benefit experience can pollute memory.

10 What Should We Do Algorithmically Next?

This view does not require training a huge foundation world model from scratch. A more practical route is a Subjective World Model adapter: wrap existing models, tools, simulators, agent runtimes, or robot policies with a measurable judgment and control structure.

In training-flow terms, the base model provides pretrained priors, VLA/WAM/tool trajectories provide task-oriented imitation, and the W2 Adapter handles credit assignment and post-training after interaction. It does not merely make the model better at imitating correct actions. It updates controllable-state judgment from feedback such as “the prediction was wrong,” “the action did not take effect,” “the test failed,” “the page is stale,” “the object slipped,” or “the experiment does not support the hypothesis.”

This adapter does a few simple but important things. It records current judgment, not only raw execution logs. It predicts action effects, not only executes actions. It estimates goal progress and failure risk, not only final success. It checks whether external memory and skills still apply to current state, rather than reusing them blindly. It writes verified experience as knowledge artifacts with source, scope, and expiry conditions, rather than treating all history as knowledge.

In engineering terms, this adapter may include heads for state judgment, action effect, failure risk, memory trust, and experience distillation. It does not have to replace the base model. It adds a checkable middle judgment layer around the base model’s observation, action, and memory.

If the task enters physical-signal or scientific-discovery settings, this layer must explicitly handle measurement trust and experimental intervention. It should ask: which instrument produced this reading, whether calibration is reliable, whether timestamps and sampling frequencies match, which variables can be actively changed, which experiment best separates two hypotheses, and which discoveries can be written as equations, causal graphs, verification scripts, or experimental protocols. In other words, action is not only task completion. It can also be an experiment that tests a world model.

For robots, the point becomes more concrete. The goal is not to train a larger robot foundation model or video model than $\pi 0$ or WorldVLA from scratch. The point is to use those models as action substrates, training signals, or strong controls, and train a W2 layer above or beside them. It should predict compact action variables rather than complete video: what state the object is in, which stage the task is in, which actions are executable under the current body-object configuration, what an action will change, where failure risk lies, whether recovery should retry, roll back, or switch strategy, and how trustworthy the simulator or world model is in this scene.

Evaluation cannot stop at success rate. In embodied settings, we should compare same-budget systems such as imitation or diffusion policies, pure VLA, pure WAM, VLA plus task-stage judgment, VLA plus W2 without skill reuse, and VLA plus W2 with skill reuse. Then we should ablate action-effect prediction and W3 distillation. The real question is whether the system has fewer repeated failures, fewer human takeovers, faster recovery, and lower future task cost under layout changes, object changes, occlusion, long-horizon tasks, cross-embodiment transfer, and sim-to-real shift. LIBERO, RoboCasa, and BEHAVIOR are suitable for this validation ladder, but only when they record object state, task stage, failure recovery, and reuse gain. [29–31]

The same structure applies to code and browser agents. In code, the trace should record hypotheses, failed commands, test outputs, patch diffs, and later reuse. In browsers, it should record page state, permission changes, whether a button actually worked, pop-ups, and recovery paths. The shared shift is from training for one correct output to training for what an action changed, how errors were repaired, and how experience reduced future cost.

Validation should also scale gradually. Start with grid worlds and clean trace logging. Move to ARC-like hidden-rule tasks to test whether the system can discover hidden rules through interaction and transfer them. Then move to digital environments such as browsers and software, where page state changes, tool side effects, permission errors, and page staleness matter. Then move to code and terminal tasks, where tests, patches, and verifiers can become external reusable knowledge. Later, add small scientific tasks: continuous-time trajectories, noisy measurements, instrument bias, intervenable variables, and hidden dynamics. Only then move to embodied environments, where objects, stages, affordances, and body state make action costly and partially observed. [26–28]

11 Scaling: Grow the Verified Experience Loop

Scaling here does not only mean bigger models, longer context, or more realistic simulators. These are useful, but if the system still does not know which states to keep, which experience is stale, and which actions change outcomes, scale may amplify noise.

One more concrete analogy is to view next-generation acting agents as a three-stage training problem:

Analogy stage	Embodied/video counterpart	What it learns	What is still missing
Pretraining	VLMs, video models, physical world models, scientific foundation models	Visual semantics, spatial structure, physical or scientific priors, common dynamics	Which variables are controllable under the current task, and which details are worth retaining
Instruction tuning	VLA, WAM, demonstration trajectories, tool-call trajectories	Instruction-to-action, observation-to-action, and action-to-future mappings	May learn imitation and correlation without learning failure causes or recovery paths

Analogy stage	Embodied/video counterpart	What it learns	What is still missing
Reinforcement-learning-style post-training	W2Trace, verifiers, failure recovery, ArtifactStore, CCC	Controllable variables, model trust, recovery policy, and experience reuse from interaction feedback	Needs closed-loop data that can be recorded, replayed, and compared

Here “reinforcement-learning-style” does not mean that only one specific RL algorithm can be used. It means the training pressure comes from what happens after interaction: whether the predicted action effect occurred, whether goal progress improved, whether failure was recognized, whether recovery became shorter, and whether experience reduced future task cost. In other words, W2 should not optimize “looks like expert action.” It should optimize “under the same budget, fewer repeated failures, faster recovery, less human takeover, and higher reuse gains on hidden tasks.”

In embodied AI, scaling should not only mean collecting more successful demonstrations or generating more synthetic video. Those are useful, but they mainly expand W1 experience supply. W2 cares about another layer of scaling: broader failure types, more complete before/after object state, recorded human takeover reasons, calibrated sim-to-real differences, and successful experience compressed into scoped skills. A robot dataset that only becomes larger is closer to a larger imitation corpus. A dataset that records these fields begins to become a trainable, auditable, compounding experience system.

For example, a video model may know that “a cup pushed to the edge of a table will fall.” That is a useful pretrained prior. A WAM may learn that “after the gripper moves toward the cup, the future frame shows the cup closer to the gripper.” That is already action-conditioned. W2 should also record whether the gripper actually touched the cup, whether the cup slipped, whether the current grasp is reliable, whether failure came from occlusion, pose, friction, or force, and whether next time the system should retry, change the grasp point, or move an obstacle first. If this failure later becomes a checklist item or recovery skill and reduces repeated failure in a new layout, it begins to generate capability compounding.

Digital tasks follow the same pattern. Browser or code-agent pretraining teaches the system roughly what web pages, code, and tests are. Tool trajectories or demonstrations teach it to click, edit, and run commands. W2 post-training should teach it that old buttons can become invalid after page refresh, which test verified which hypothesis, why a patch route failed, and which experience still applies to the current repository. The key data is not the full chat transcript. It is pre-action and post-action state, predicted outcome, real delta, failure category, recovery action, and later reuse gain.

Likewise, scaling scientific world models should not only mean “feed more raw signals.” Raw signals are not reality itself; they still pass through sensors, instruments, and sampling chains. What matters is putting calibrated measurements, continuous-time trajectories, intervention records, uncertainty, error models, and testable scientific artifacts into the same closed loop. Otherwise the system may simply move from a JPEG-and-text world to a sensor-reading world without learning which variables are controllable, which laws transfer, and which conclusions can be experimentally falsified.

Therefore, the object of scaling is not a single internal representation. It is the whole post-training loop: more open interactive worlds, more precise current judgment, more reliable external knowledge artifacts, and stricter hidden-task validation. Only when all four scale together can experience become capability compounding. Otherwise the system may become better at imagining, imitating, and executing, but not necessarily better at learning from failure and reuse.

Scaling subjective world models: grow the verified experience loop

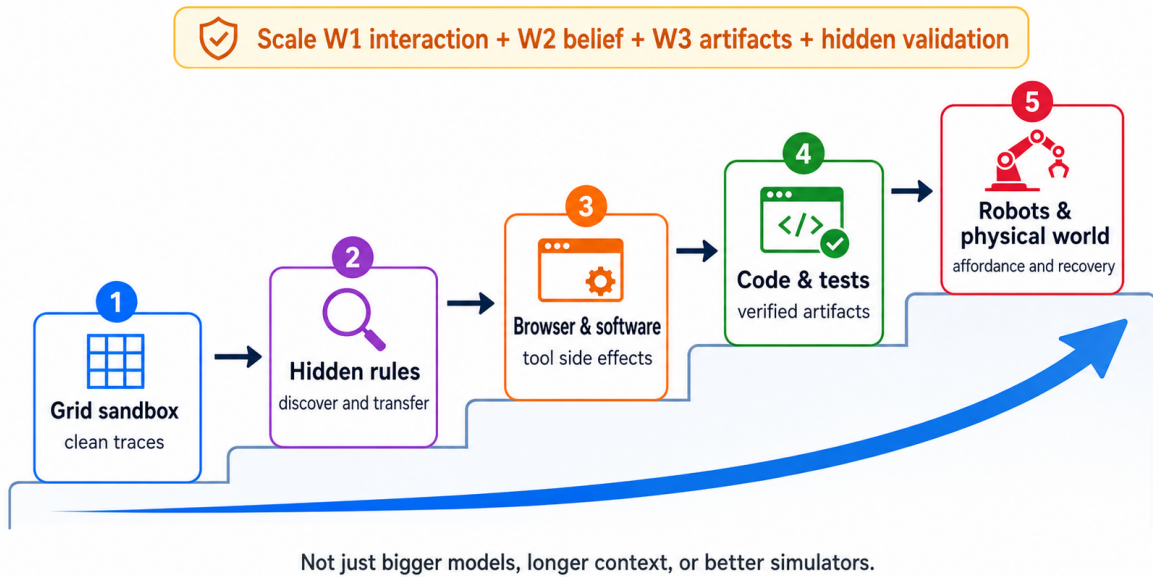


Figure 6: Scaling subjective world models: from grid sandboxes to robots, each level must expand the verified experience loop.

12 Counterargument: Maybe W2 Emerges Implicitly

This essay does not say every strong system must contain an explicit module named W2. The strongest counterargument is simple: maybe scale, longer context, more interaction data, stronger post-training, or larger WAM systems will learn the same function implicitly.

That is possible.

If a system without an explicit W2 adapter can match the explicit adapter under the same model, data, tool, interaction, and validation budgets on recovery, transfer, external knowledge reuse, and hidden-task CCC, then explicit W2 should not be claimed as necessary. In embodied settings, this should be tested seriously: if a larger WAM system can reliably handle occlusion, layout change, failure recovery, cross-embodiment transfer, lower human takeover, and experience reuse under the same budget, then the necessity of a W2 adapter should be weakened. It may be only an analysis framework, or useful only in some domains.

But if bypass routes succeed locally while repeatedly showing the same failure modes, such as trusting stale memory, confusing correlation with controllability, repeating errors after failure, or failing to reduce future cost, then W2 is not just a label. It becomes a missing interface in current systems.

13 Conclusion: From World Simulation to Experience Compounding

The term world model is confusing because it covers many implementations: reinforcement learning models, video generation models, robot state-action policies, agent reasoning, planning, memory, tool traces, tests, and skills. It is useful to place them under one broad frame: they all process future-related knowledge.

But the next question is narrower: what future knowledge should a finite agent keep? The Tri-World Model answers by separating interaction, internal judgment, and external knowledge. The Interactive Objective World provides environments that respond to action. The External

Knowledge World preserves reusable artifacts. The Subjective World Model selects controllable, recoverable, reusable future information under the current goal. Pretraining can provide broad priors, and demonstration learning can provide common action patterns, but long-term capability compounding also needs a post-training loop that keeps updating from interaction, failure, verification, and reuse.

So the value of W2 is not the name. Its value is a testable standard: if bypass routes can achieve the same recovery, transfer, knowledge reuse, and capability compounding under the same budget, explicit W2 is unnecessary. If they cannot, W2 is the key layer next-generation agents need to add.

References

- [1] Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi: 10.1007/BF00115009. URL <https://link.springer.com/article/10.1007/BF00115009>.
- [2] David Ha and Jürgen Schmidhuber. World Models. arXiv preprint arXiv:1803.10122, 2018. URL <https://arxiv.org/abs/1803.10122>.
- [3] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4. URL <https://www.nature.com/articles/s41586-020-03051-4>.
- [4] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering Diverse Domains through World Models. arXiv preprint arXiv:2301.04104, 2023. URL <https://arxiv.org/abs/2301.04104>.
- [5] Yann LeCun. A Path Towards Autonomous Machine Intelligence. OpenReview position paper, 2022. URL <https://openreview.net/forum?id=BZ5a1r-kVsf>.
- [6] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting Feature Prediction for Learning Visual Representations from Video. arXiv preprint arXiv:2404.08471, 2024. URL <https://arxiv.org/abs/2404.08471>.
- [7] OpenAI. Video generation models as world simulators. Technical report, 2024. URL <https://openai.com/research/video-generation-models-as-world-simulators>.
- [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629, 2022. URL <https://arxiv.org/abs/2210.03629>.
- [9] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv preprint arXiv:2303.11366, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [10] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv preprint arXiv:2305.16291, 2023. URL <https://arxiv.org/abs/2305.16291>.
- [11] AGI Should Not Only Read Text: From Physical Signals to Scientific World Models. WeChat essay, 2026. URL https://mp.weixin.qq.com/s/bHftgHJZ30aV0P2_xfsA0A. Chinese-language source; English title supplied in the blog.
- [12] Meta AI. Introducing the V-JEPA 2 world model and new benchmarks for physical reasoning. Technical blog, 2025. URL <https://ai.meta.com/blog/v-jepa-2-world-model-benchmarks/>.
- [13] Archetype AI. Physical AI / Newton. Product and research overview, 2026. URL <https://www.archetypeai.io/physical-ai>. Accessed 2026-09-06.
- [14] Jaime Lien, Laura I. Galindez Olascoaga, Hasan Dogan, Nicholas Gillian, Brandon Barbello, Leonardo Giusti, and Ivan Poupyrev. A Phenomenological AI Foundation Model for Physical Signals. arXiv preprint arXiv:2410.14724, 2024. URL <https://arxiv.org/abs/2410.14724>.

- [15] Liam Parker, Francois Lanusse, Jeff Shen, Ollie Liu, Tom Hehir, Leopoldo Sarra, et al. AION-1: Omnimodal Foundation Model for Astronomical Sciences. arXiv preprint arXiv:2510.17960, 2025. URL <https://arxiv.org/abs/2510.17960>.
- [16] Michael McCabe, Payel Mukhopadhyay, Tanya Marwah, Bruno Regaldo-Saint Blancard, Francois Rozet, Cristiana Diaconu, et al. Walrus: A Cross-Domain Foundation Model for Continuum Dynamics. arXiv preprint arXiv:2511.15684, 2025. URL <https://arxiv.org/abs/2511.15684>.
- [17] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data: Sparse identification of nonlinear dynamical systems. arXiv preprint arXiv:1509.03580, 2015. URL <https://arxiv.org/abs/1509.03580>.
- [18] Bernhard Schölkopf, Francesco Locatello, Stefan Bauer, Nan Rosemary Ke, Nal Kalchbrenner, Anirudh Goyal, and Yoshua Bengio. Towards Causal Representation Learning. arXiv preprint arXiv:2102.11107, 2021. URL <https://arxiv.org/abs/2102.11107>.
- [19] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv preprint arXiv:2204.01691, 2022. URL <https://arxiv.org/abs/2204.01691>.
- [20] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv preprint arXiv:2307.15818, 2023. URL <https://arxiv.org/abs/2307.15818>.
- [21] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, et al. OpenVLA: An Open-Source Vision-Language-Action Model. arXiv preprint arXiv:2406.09246, 2024. URL <https://arxiv.org/abs/2406.09246>.
- [22] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control. arXiv preprint arXiv:2410.24164, 2024. URL <https://arxiv.org/abs/2410.24164>.
- [23] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, et al. WorldVLA: Towards Autoregressive Action World Model. arXiv preprint arXiv:2506.21539, 2025. URL <https://arxiv.org/abs/2506.21539>.
- [24] Karl Popper. Three Worlds. Tanner Lecture, University of Michigan, April 7, 1978. URL <https://tannerlectures.org/lectures/three-worlds/>.
- [25] Stephen Thornton. Karl Popper. Online publication, 1997. URL <https://plato.stanford.edu/entries/popper/>.
- [26] François Chollet et al. ARC-AGI-3 Technical Report. ARC Prize technical report, 2026. URL https://arcprize.org/media/ARC_AGI_3_Technical_Report.pdf.
- [27] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, et al. OS-World: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv preprint arXiv:2404.07972, 2024. URL <https://arxiv.org/abs/2404.07972>.
- [28] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv preprint arXiv:2310.06770, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [29] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. arXiv preprint arXiv:2306.03310, 2023. URL <https://arxiv.org/abs/2306.03310>.
- [30] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots. arXiv preprint arXiv:2406.02523, 2024. URL <https://arxiv.org/abs/2406.02523>.
- [31] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, et al. BEHAVIOR-1K: A Human-Centered, Embodied AI Benchmark with 1,000 Everyday Activities and Realistic Simulation. arXiv preprint arXiv:2403.09227, 2024. URL <https://arxiv.org/abs/2403.09227>.